UniSmart AI

Descripción detallada del proyecto

Concepto, funcionalidades, arquitectura, herramientas, base de datos, inteligencia artificial, organización del equipo y plan de trabajo de ocho semanas.

Documento de planificación para el desarrollo de un asistente universitario inteligente.

1. ¿Qué es UniSmart AI?

UniSmart AI es una aplicación web que funcionará como un asistente inteligente para la comunidad universitaria. Su objetivo es reunir en un solo sitio información académica y administrativa que normalmente está dispersa en documentos, avisos, horarios y diferentes áreas de la institución. El estudiante podrá consultar información mediante una conversación en lenguaje natural, sin necesidad de crear una cuenta ni iniciar sesión.

El sistema combinará una interfaz web, una base de datos con información organizada y un modelo de inteligencia artificial que ayude a interpretar las preguntas y explicar la información encontrada. La IA no debe ser la única fuente de verdad: los datos importantes, como horarios, aulas, trámites vigentes y estados de reportes, deberán almacenarse y consultarse desde fuentes controladas.

Además de la parte pública para estudiantes, existirá un panel administrativo protegido. El personal autorizado iniciará sesión y solo podrá gestionar las áreas que tenga asignadas. Por ejemplo, una persona responsable de documentos podrá publicar o actualizar documentos, pero no necesariamente modificar horarios o administrar a todos los usuarios.

2. Problema que busca resolver

En muchas universidades, los estudiantes tienen que buscar información en distintos documentos PDF, páginas, avisos o preguntar directamente a diferentes oficinas. Esto puede provocar pérdida de tiempo, confusión entre versiones de documentos y errores al interpretar horarios, salones o procedimientos.

UniSmart AI busca facilitar el acceso a esa información desde una sola plataforma, permitir consultas escritas en español cotidiano y ofrecer respuestas respaldadas por datos y documentos de la institución. También permitirá canalizar reportes de mantenimiento y consultar ubicaciones del campus.

3. Objetivos del proyecto

- Objetivo general: desarrollar un asistente web universitario que centralice información académica y administrativa y facilite su consulta mediante inteligencia artificial.
- Objetivos específicos: responder preguntas en lenguaje natural; consultar horarios estructurados; localizar documentos y explicar trámites; mostrar un mapa virtual del campus; recibir reportes de mantenimiento; ofrecer un panel de administración con permisos por área; y registrar cambios importantes para mantener trazabilidad.

4. Usuarios y tipos de acceso

4.1 Estudiantes y visitantes

- Accederán a la página pública sin registro ni inicio de sesión.
- Podrán hacer preguntas al chatbot y consultar información pública autorizada.
- Podrán buscar horarios por grupo, materia, docente o aula, siempre que esos datos estén disponibles.
- Podrán consultar documentos, procedimientos y ubicaciones del campus.
- Podrán enviar reportes de mantenimiento mediante un formulario público, con controles para reducir abuso y spam.

4.2 Personal administrativo

El personal administrativo iniciará sesión en un panel privado. Sus permisos dependerán de los roles y áreas asignados. El sistema deberá validar los permisos en el backend en cada operación, no solo ocultar botones en la interfaz.

4.3 Administrador del sistema

Un administrador autorizado podrá crear usuarios, asignar roles y áreas, activar o desactivar cuentas y revisar registros de auditoría. Esta capacidad debe limitarse a las personas responsables de administrar el sistema.

5. Funcionalidades principales

5.1 Chatbot en lenguaje natural

El chatbot permitirá preguntas como: “¿En qué salón toma clases el grupo 3AS?”, “¿Qué documentos necesito para un trámite?” o “¿Dónde se encuentra un edificio?”. No estará limitado a botones o preguntas predefinidas: deberá interpretar preguntas formuladas de diferentes maneras y pedir aclaraciones cuando falte información.

Las respuestas relacionadas con información institucional deberán basarse en registros o documentos disponibles. Cuando no encuentre evidencia suficiente, el sistema deberá decirlo con claridad, en lugar de inventar una respuesta. Siempre que sea posible, mostrará el nombre del documento, la página o la fuente consultada.

5.2 Documentos y trámites

El personal autorizado podrá cargar y actualizar documentos institucionales, por ejemplo reglamentos, calendarios, formatos, convocatorias y guías de trámites. El sistema extraerá el texto, lo dividirá en fragmentos manejables y permitirá recuperar las partes pertinentes para responder preguntas.

5.3 Horarios y datos académicos

Los horarios deben guardarse como datos estructurados en la base de datos, no depender únicamente de que el modelo de IA lea un PDF cada vez. Se podrán consultar por grupo, materia, docente, aula y periodo escolar. Para importar información desde PDF o Excel, se recomienda una plantilla CSV/Excel validada y una revisión humana antes de publicar.

Como referencia del archivo de horarios revisado durante la planeación, el periodo indicado es agosto-diciembre de 2026; los grupos mencionados incluyen 1AS en el aula 15, 3AS en el aula 16, 5AS en el aula 17 y 7AS en el aula 18. Estos datos deben verificarse con el documento oficial vigente antes de incorporarlos al sistema.

5.4 Mapa virtual del campus

El mapa permitirá visualizar edificios, aulas, oficinas, accesos y otros puntos relevantes. Si existe un plano oficial, puede usarse como base, por ejemplo mediante una imagen o SVG interactivo. Si se dispone de coordenadas y una cartografía apropiada, puede utilizarse Leaflet con OpenStreetMap. Es importante verificar que los datos y las licencias del mapa sean adecuados para el uso previsto.

5.5 Reportes de mantenimiento

Los estudiantes podrán reportar problemas como luminarias dañadas, mobiliario roto, fugas, problemas de limpieza o desperfectos en un espacio. El formulario puede solicitar ubicación, tipo de problema y descripción. El sistema generará un folio para confirmar la recepción y el personal autorizado podrá cambiar el estado del reporte.

Los estados pueden incluir recibido, en revisión, asignado, en proceso y resuelto. El historial de cambios permitirá saber cuándo se actualizó un reporte. Si los reportes se consultan públicamente, se deben ocultar datos personales y evitar exponer información interna.

5.6 Panel de administración

El panel permitirá gestionar usuarios, roles, áreas, documentos, horarios, ubicaciones y reportes, según los permisos concedidos. Debe incluir formularios de validación, mensajes de error claros y confirmaciones para acciones importantes.

6. Herramientas y tecnologías recomendadas

La selección busca mantener el proyecto accesible, viable en Windows y basada en herramientas gratuitas o de código abierto siempre que sea posible.

- Frontend: React, para construir la página pública, el chat, los formularios y el panel administrativo. Puede usarse Vite para iniciar el proyecto.
- Backend: Python con FastAPI, para crear una API que atienda las solicitudes del frontend, consulte la base de datos y controle los permisos.
- Base de datos: PostgreSQL, para guardar usuarios, roles, horarios, documentos, ubicaciones y reportes.
- IA local: LM Studio con Qwen3-4B Instruct, para ejecutar el modelo localmente durante desarrollo y pruebas y acceder a él mediante su API compatible.
- Extracción de PDF: PyMuPDF, para leer texto de documentos digitales. Si los PDF son escaneados, puede ser necesario usar OCR.
- OCR: Tesseract, para extraer texto de páginas que contienen imágenes, con revisión posterior porque el OCR puede cometer errores.
- Recuperación de documentos (RAG): un proceso de búsqueda que recupera fragmentos pertinentes de documentos antes de generar una respuesta. Se puede evaluar PostgreSQL con pgvector para almacenar vectores.
- Mapa: Leaflet y OpenStreetMap, o un plano oficial de la institución si resulta más preciso.
- Control de versiones: Git y GitHub, para trabajar en equipo, revisar cambios y recuperar versiones anteriores.
- Pruebas: Pytest para el backend y Postman o Bruno para probar la API.
- Despliegue: Docker Compose puede ayudar a levantar servicios de manera reproducible, aunque no es obligatorio para el primer prototipo.

7. Cómo funcionará la inteligencia artificial

7.1 Modelo Qwen3-4B Instruct

El modelo seleccionado para comenzar es Qwen3-4B Instruct, ejecutado mediante LM Studio. La computadora principal indicada cuenta con Windows, 24 GB de RAM y gráficos integrados Radeon 880M. Esta configuración puede ser suficiente para experimentar con un modelo cuantizado y construir un prototipo, pero el rendimiento real dependerá de la cuantización, el contexto utilizado, la memoria compartida disponible y la carga de trabajo. Deben hacerse pruebas con preguntas reales antes de fijar expectativas de velocidad.

LM Studio es adecuado para desarrollo local. No se debe exponer directamente su servidor de inferencia a internet público sin una arquitectura de seguridad y despliegue apropiada. Para producción, la universidad necesitará un servidor institucional autorizado o una alternativa de inferencia aprobada.

7.2 RAG: recuperación aumentada por generación

Para responder preguntas basadas en documentos, se recomienda un flujo RAG. La idea es recuperar información relevante y entregársela al modelo como contexto, en vez de pedirle que recuerde o invente datos.

1. Un administrador autorizado carga un documento.
2. El backend valida el archivo y extrae el texto; si es escaneado, puede pasar por OCR.
3. El texto se divide en fragmentos y cada fragmento conserva metadatos como nombre del documento, página, fecha y área responsable.
4. Se generan representaciones vectoriales (embeddings) y se guardan junto con el texto, o se utiliza un mecanismo de búsqueda apropiado.
5. Cuando una persona pregunta algo, el sistema busca los fragmentos más relevantes.
6. El backend envía la pregunta y los fragmentos encontrados a Qwen mediante LM Studio.
7. La respuesta se presenta con referencias al documento y página cuando sea posible. Si la evidencia es insuficiente, el chatbot debe indicarlo.

El RAG no garantiza por sí solo respuestas perfectas. Es necesario probar la recuperación, limitar las respuestas a la evidencia disponible y evaluar preguntas cuya respuesta se conozca.

8. Por qué no conviene depender del PDF directamente para los horarios

Durante las pruebas con el horario escolar, Qwen no interpretó correctamente toda la información del PDF: no identificó de forma fiable el salón y el horario de un grupo y llegó a asociar un aula incorrecta. Esto demuestra que el problema puede estar en la extracción del PDF, en cómo se divide el contenido, en el contexto enviado al modelo o en la capacidad del modelo para interpretar tablas.

La solución recomendada no es entrenar el modelo de inmediato. Primero conviene probar el PDF por páginas, extraer el texto con una herramienta como PyMuPDF, revisar el resultado y probar un archivo TXT limpio. Para los datos exactos —horarios, salones, materias y grupos— se debe usar una base de datos estructurada. La IA puede interpretar la pregunta y explicar el resultado, pero la base de datos debe ser la fuente de verdad.

No se recomienda comenzar con ajuste fino o entrenamiento propio: implica más trabajo y no corrige necesariamente una extracción defectuosa o datos mal estructurados.

9. Diseño de la base de datos

La base de datos debe diseñarse de forma que los datos estén relacionados y puedan actualizarse sin duplicaciones innecesarias. Las tablas iniciales sugeridas son:

- users: cuentas del personal administrativo, estado de la cuenta y datos de autenticación.
- roles: tipos de permisos, como administrador, editor de documentos o responsable de mantenimiento.
- areas: departamentos o áreas de responsabilidad.
- user_roles: relación entre usuarios y roles, con el alcance de área correspondiente.
- documents: nombre, tipo, versión, fecha, responsable, estado y ubicación del archivo.
- document_chunks: fragmentos de texto, página y metadatos utilizados para buscar información.
- groups: grupos académicos, como 1AS, 3AS, 5AS y 7AS.
- subjects: materias o asignaturas.
- teachers: docentes, si la institución autoriza almacenar y publicar esos datos.
- classrooms: aulas y sus ubicaciones.
- schedule_entries: día, hora de inicio y fin, grupo, materia, docente, aula y periodo escolar.
- maintenance_reports: folio, ubicación, categoría, descripción, estado y fechas.
- report_history: cambios de estado y acciones relacionadas con cada reporte.
- campus_locations: edificios, oficinas, aulas, coordenadas o referencias al plano.
- audit_logs: registro de acciones administrativas importantes.

Las relaciones exactas, claves primarias, claves foráneas, índices y restricciones se definirán durante el diseño técnico. Deben incluirse validaciones para evitar horarios duplicados o inconsistentes.

10. Seguridad y privacidad

- La parte pública no debe mostrar información privada del personal ni datos internos de los reportes.
- Las contraseñas de administradores deben almacenarse con un algoritmo de hash apropiado; nunca como texto plano.
- La autorización debe verificarse en el backend en cada operación administrativa.
- Los archivos cargados deben validarse por tipo, tamaño y contenido; no se debe confiar solo en la extensión.
- Se deben prevenir inyecciones SQL mediante consultas parametrizadas u ORM.
- Conviene limitar solicitudes, controlar spam en reportes públicos y registrar errores sin almacenar secretos.
- Se deben hacer respaldos de la base de datos y de los documentos.
- En producción, utilizar HTTPS y gestionar secretos mediante variables de entorno o un mecanismo seguro.
- Los registros de auditoría deben permitir revisar quién modificó información importante y cuándo.

11. Organización de los cuatro equipos

Se propone dividir el trabajo en cuatro equipos, con responsables definidos y una integración frecuente. Los equipos no deben desarrollar componentes aislados hasta el final: conviene acordar desde el principio los contratos de la API, los modelos de datos y los formatos de respuesta.

Equipo 1: IA y conocimiento

- Configurar LM Studio y Qwen3-4B Instruct.
- Probar preguntas reales en español y registrar errores.
- Preparar extracción de PDF, fragmentación, búsqueda RAG y citas de fuentes.
- Definir cuándo responder, cuándo pedir aclaraciones y cuándo reconocer que no hay información.

Equipo 2: Frontend

- Construir la página pública y la interfaz del chatbot.
- Diseñar vistas para horarios, documentos, mapa y reportes.
- Crear el panel administrativo y formularios de gestión.
- Asegurar que la interfaz sea usable en móviles y computadoras.

Equipo 3: Backend y base de datos

- Diseñar PostgreSQL y las relaciones de las tablas.
- Crear la API con FastAPI.
- Implementar autenticación y permisos por rol y área.
- Construir endpoints para horarios, documentos, reportes, usuarios y ubicaciones.

Equipo 4: Integración, pruebas y despliegue

- Definir casos de prueba y probar la API y la interfaz.
- Verificar permisos, seguridad, calidad de las respuestas y comportamiento ante datos faltantes.
- Integrar cambios de los demás equipos mediante Git.
- Preparar documentación, demostración y despliegue de prueba.

La computadora principal puede encargarse de las pruebas de IA y apoyar la integración. Claude Pro/Claude Code puede ayudar con tareas delimitadas —por ejemplo, generar pruebas, proponer endpoints o revisar un módulo—, pero el equipo debe revisar, probar y comprender el código antes de integrarlo. No se recomienda entregar a una herramienta de IA cambios amplios sin criterios de aceptación ni revisión.

12. Plan de trabajo de ocho semanas

Semana 1: definición y prueba de viabilidad

Acordar el alcance mínimo, crear el repositorio Git, configurar el entorno de desarrollo, probar Qwen en LM Studio con preguntas reales y reunir documentos institucionales autorizados. Documentar qué preguntas responde correctamente y cuáles fallan.

Semana 2: base de datos y API inicial

Diseñar las tablas iniciales de PostgreSQL, crear el proyecto FastAPI y preparar una primera interfaz web. Definir los contratos entre frontend y backend.

Semana 3: conexión con la IA

Conectar FastAPI con la API local de LM Studio. Implementar una primera versión del chat, manejo de errores, límites de contexto y respuestas cuando el modelo no esté disponible.

Semana 4: documentos, RAG y horarios

Implementar extracción de texto de PDF, fragmentación y búsqueda de documentos. Importar horarios a tablas estructuradas y validar manualmente los datos antes de publicarlos.

Semana 5: panel administrativo y permisos

Implementar inicio de sesión administrativo, gestión de usuarios, asignación de áreas y control de permisos. Probar que un usuario no pueda modificar información de otra área.

Semana 6: mantenimiento y mapa

Crear el formulario público de reportes, folios e historial de estados. Integrar el mapa del campus con ubicaciones verificadas.

Semana 7: integración, pruebas y seguridad

Probar los flujos completos, corregir errores, revisar respuestas del chatbot, validar archivos y permisos, y probar respaldos y recuperación.

Semana 8: despliegue y documentación

Preparar una demostración estable, guía de instalación, manual de usuario y manual administrativo. Realizar pruebas de aceptación con un conjunto de preguntas y operaciones previamente definido.

Este calendario es exigente para cuatro equipos y depende de que el alcance se mantenga controlado. Si surgen retrasos, priorizar el chatbot con documentos, horarios fiables y administración básica; dejar funciones avanzadas para una siguiente versión.

13. Alcance recomendado para la primera versión (MVP)

Para entregar un producto funcional dentro de dos meses, la primera versión debería incluir:

- Página pública sin inicio de sesión.
- Chatbot conectado a Qwen a través del backend.
- Consulta de horarios desde datos estructurados.
- Búsqueda en un conjunto pequeño y validado de documentos institucionales con referencias.
- Panel administrativo con autenticación y permisos por área.
- Carga y actualización controlada de documentos.
- Formulario de reportes de mantenimiento con folio y estado.
- Mapa básico con ubicaciones verificadas.
- Pruebas documentadas, guía de instalación y manual de uso.

Funciones como una aplicación móvil nativa, analítica avanzada, integración con sistemas institucionales existentes, fine-tuning del modelo o automatizaciones complejas pueden quedar para etapas posteriores.

14. Pruebas y criterios de aceptación

Antes de dar el sistema por terminado, el equipo debe preparar un conjunto de preguntas y tareas con respuestas verificadas. Por ejemplo:

- Preguntar por el aula de un grupo y comparar la respuesta con el registro oficial.
- Consultar una materia por día y hora y verificar que no se inventen datos.
- Preguntar por un trámite y comprobar que la respuesta cite el documento vigente.
- Preguntar algo que no aparece en los documentos y comprobar que el chatbot reconozca la falta de información.
- Intentar que un usuario administrativo modifique un área que no tiene asignada y verificar que la API lo rechace.
- Cargar un archivo no permitido y verificar que el sistema lo bloquee.
- Enviar un reporte de mantenimiento y comprobar que genere folio y registre sus cambios de estado.
- Probar la página en móvil y escritorio, así como el comportamiento si LM Studio o la base de datos no están disponibles.

Los criterios concretos —por ejemplo, porcentaje mínimo de respuestas correctas— deben acordarse antes de las pruebas finales, utilizando un conjunto representativo de preguntas y no solo ejemplos preparados para que el sistema funcione.

15. Orden recomendado para empezar

1. Confirmar el alcance y los responsables de cada equipo.
2. Probar Qwen3-4B Instruct en LM Studio con preguntas representativas y medir sus límites.
3. Reunir y clasificar documentos oficiales; comprobar permisos, versiones y fechas.
4. Convertir los horarios a una tabla estructurada y validarlos manualmente.
5. Diseñar la base de datos y definir los endpoints de la API.
6. Crear la página pública y conectar el chatbot al backend.
7. Implementar recuperación de documentos con referencias a fuentes.
8. Desarrollar el panel administrativo y los permisos por área.
9. Añadir reportes de mantenimiento y mapa del campus.
10. Integrar, probar, documentar y preparar el despliegue.

16. Conclusión

UniSmart AI es viable como prototipo universitario si se limita bien el alcance, se mantiene una fuente de datos fiable y se prueban las respuestas con información real. La arquitectura propuesta separa la interfaz, la lógica del servidor, la base de datos y el modelo de IA, lo que facilita que los equipos trabajen en paralelo.

La decisión técnica más importante es que la inteligencia artificial no sustituya a los registros oficiales. Qwen debe ayudar a interpretar las preguntas y explicar la información; PostgreSQL debe conservar los datos estructurados, y el sistema RAG debe recuperar evidencia de los documentos institucionales. Los permisos administrativos se validan en el backend y la parte pública solo expone información autorizada.

Para el plazo de ocho semanas, lo más prudente es construir primero un MVP con consultas fiables de horarios, respuestas documentales con fuentes, un panel administrativo básico y un flujo de reportes. Una

aplicación web adaptable a móviles es un mejor primer paso que intentar desarrollar simultáneamente una app nativa.

Resumen de la recomendación central: primero aseguren que los datos y las fuentes sean correctos; después utilicen la IA para facilitar su consulta. No intenten resolver problemas de extracción de documentos entrenando el modelo antes de revisar el flujo de datos.